Unit 2 – If / Else If in p5.js

Name: _____ Date: _____ Period: _____

Syntax Rules: Read Before You Start

Every if statement in p5.js has the same shape:

the keyword `if`,

the condition inside parentheses `( )`,

then the code to run inside curly braces `{ }`.

```
if (condition) {
  // runs when condition is true
} else if (otherCondition) {
  // runs when the first is false and this one is true
} else {
  // runs when everything above is false
}
```

if – open paren – condition – close paren – open brace – code – close brace.

Wrong	Why it's wrong	Correct
`if x > 400 { }`	Condition needs parentheses	
`if x > 400() { }`	Parentheses go around the condition, not after it	
`if (x > 400 { })`	Close the) before opening the {	
`if (x > 400); { }`	A semicolon ends the if; the braces always run	
`if (lives = 0) { }`	= assigns a value; === compares	
`elseif (x > 2) { }`	Two separate words	
`else (x > 2) { }`	else never has a condition	

Wrong	Why it's wrong	Correct
`if (x > 2 and y < 5)`	JavaScript uses symbols	

Logical operators: `&&` means AND (both must be true), `||` means OR (at least one is true), `!` means NOT. All questions use a 400 × 400 canvas, so `width` and `height` both equal 400. The ball's position is `x` and `y`.

Section 1: What's the Value?

Trace each block of code. Write the final value of the variable (or what prints to the console) for each starting value.

1. What is `score` after the code runs if `score` starts at: a) 1 b) 3 c) 10

```
let score = ...;
if (score > 5) {
  score = score + 5;
} else if (score > 2) {
  score = score + 10;
}
```

a. _____ b. _____ c. _____

2. Same starting values: a) 1 b) 3 c) 10. Watch out: these are two separate if statements.

```
let score = ...;
if (score > 5) {
  score = score + 5;
}
if (score > 2) {
  score = score + 10;
}
```

a. _____ b. _____ c. _____

3. Same starting values: a) 1 b) 3 c) 10

```
let score = ...;
if (score > 5) {
  score = score + 5;
} else if (score > 2) {
  score = score + 10;
} else {
  score = score + 15;
}
```

a. _____ b. _____ c. _____

4. Wall bounce. What is `speedX` after the code runs if `x` is: a) 200 b) 395 c) 5

```
let speedX = 4;
let x = ...;
if (x > width - 10 || x < 10) {
  speedX = speedX * -1;
}
```

a. _____ b. _____ c. _____

5. Random power-up. `random(1)` gives a decimal from 0 up to (not including) 1.

```
let r = random(1);
let powerUp;
if (r < 0.25) {
  powerUp = "BIG PADDLE";
} else if (r < 0.5) {
  powerUp = "SLOW BALL";
} else {
  powerUp = "NONE";
}
```

a. If `r` is 0.2, what is `powerUp`? _____

b. If `r` is 0.35, what is `powerUp`? _____

c. About what percent of the time will the player get "NONE"? _____

6. Level messages. What prints to the console if `score` is: a) 38 b) -4 c) 99

```
let score = ...;
if (score < 13) {
  console.log("Level 1");
} else if (score < 20) {
  console.log("Level 2");
} else if (score < 40) {
  console.log("Level 3");
}
console.log("Keep playing!");
```

a. _____

b. _____

c. _____

7. Paddle catch (nested if). The paddle is at `paddleX = 100` and is 80 pixels wide. Start values: `speedY = 5`, `lives = 3`. Give `speedY` and `lives` after the code runs for each ball position.

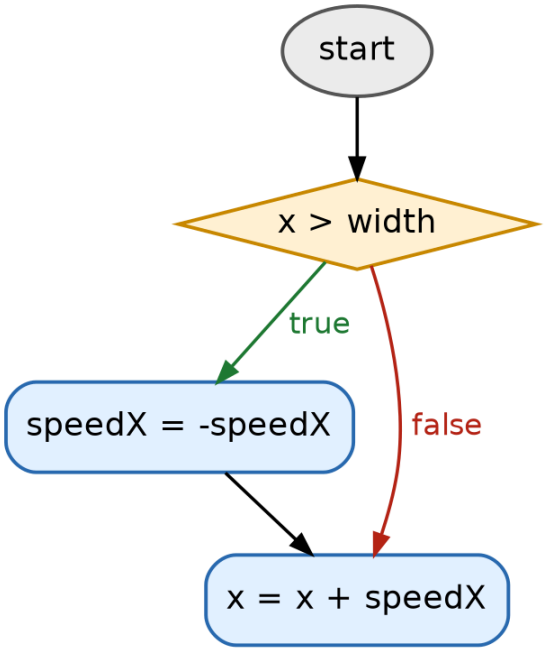
```
if (y >= 380) {
  if (x > paddleX && x < paddleX + 80) {
    speedY = -speedY;
  } else {
    lives = lives - 1;
  }
}
```

	x	y	speedY	lives
a.	150	200		
b.	150	385		
c.	220	385		

Section 2: Flowchart to Code

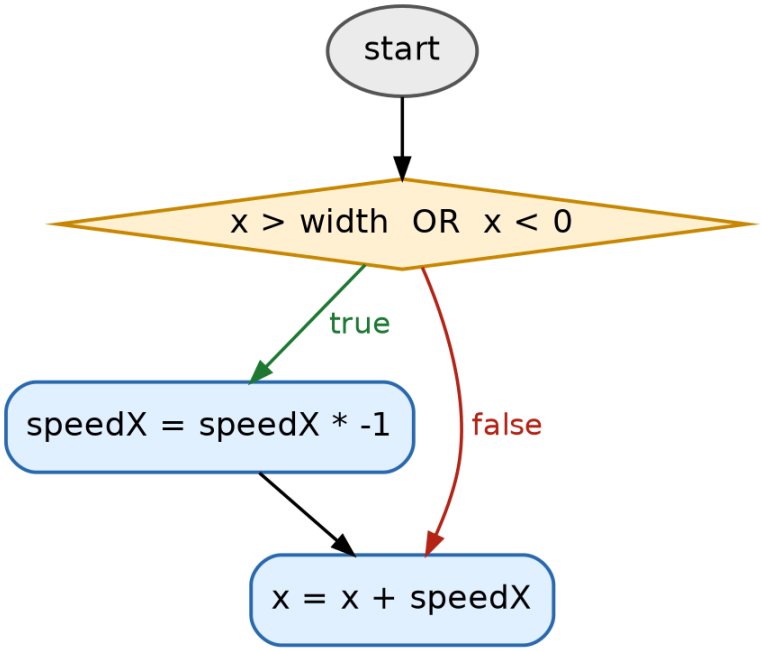
Write the p5.js code for each flowchart. Each diamond is a condition; each box is a statement. Follow the green true and red false arrows to decide whether you need `if`, `else if`, `else`, or two separate `if` statements.

8. Bounce off the right wall



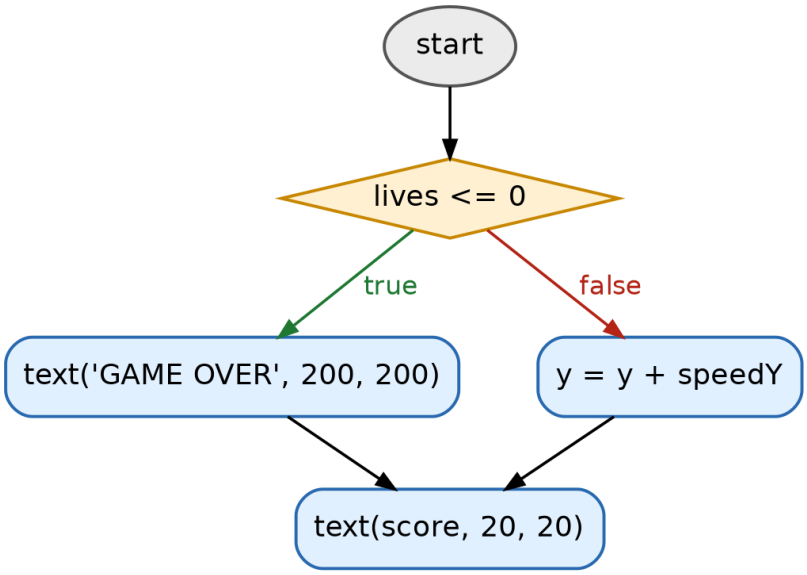
---

9. Bounce off both side walls



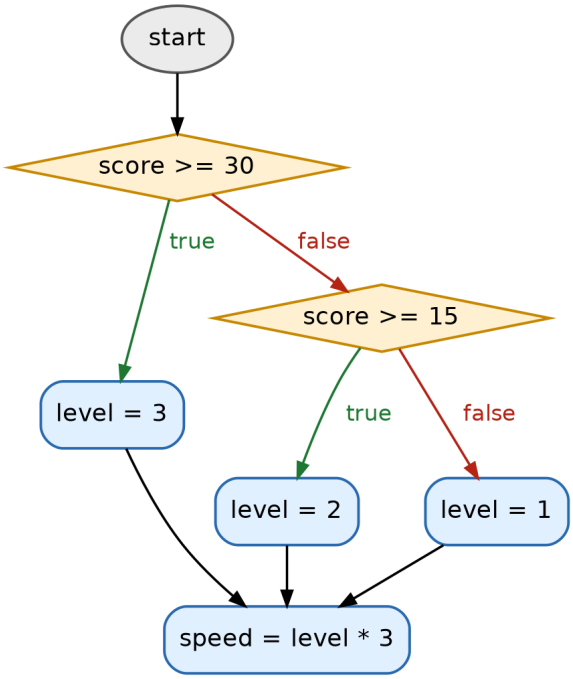
---

10. Game over or keep moving

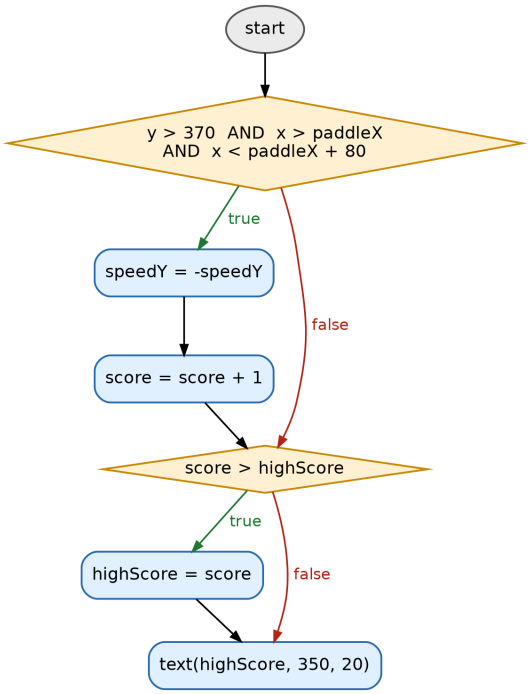


---

11. Set the level from the score

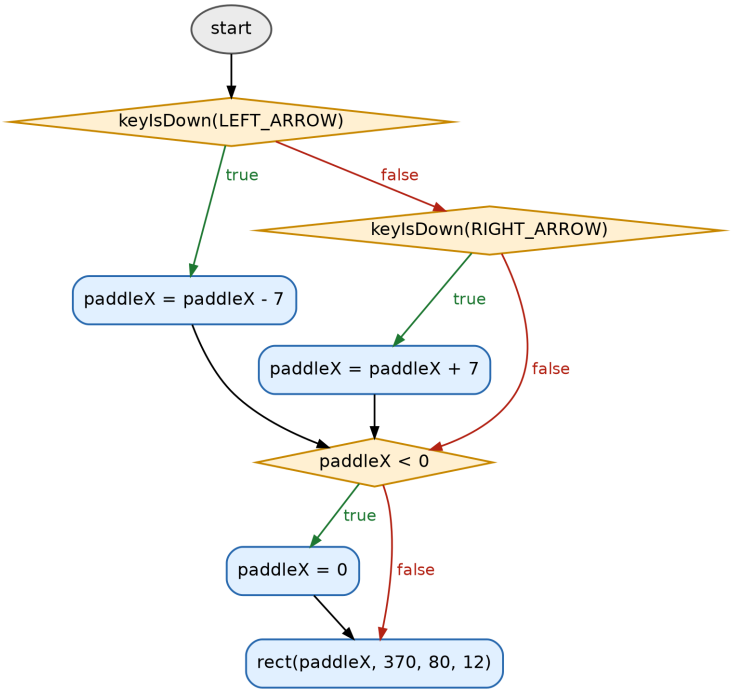


12. Paddle hit, then high score



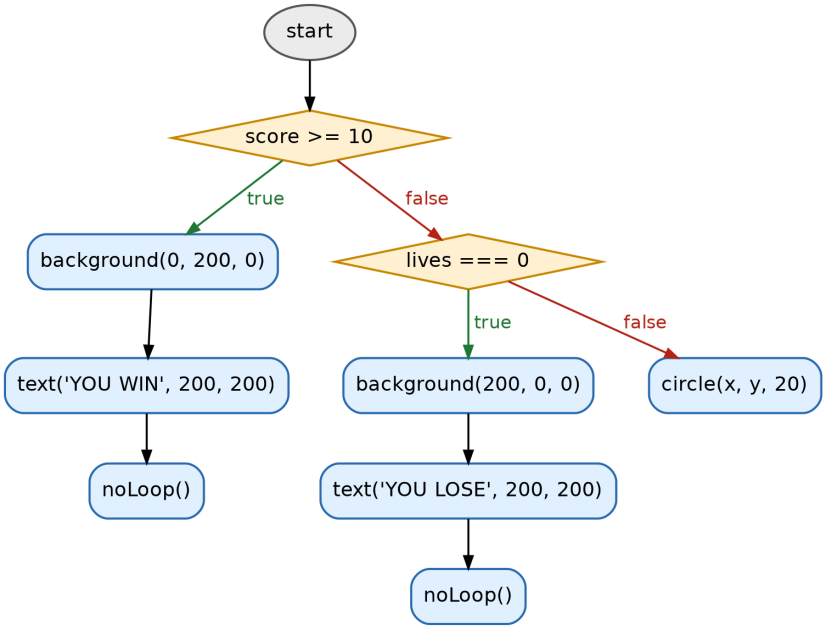
---

13. Move the paddle and keep it on screen



---

14. Win, lose, or keep playing



Section 3: Write It Yourself

Write each if statement by hand. Use the variable names given. Check every answer against the Syntax Rules table: parentheses around the condition, braces around the code, `===` to compare.

Variables available: `x`, `y` (ball position), `speedX`, `speedY`, `ballSize` (20), `paddleX`, `paddleY` (370), `paddleW` (80), `score`, `lives`, `level`, `gameState`, `targetX`, `targetY`.

15. Top wall bounce. If the ball's `y` is less than 10, reverse its vertical direction.

16. Missed the ball. If `y` is greater than `height`, subtract 1 from `lives`, then put the ball back at the center of the canvas (x = 200, y = 200).

17. Paddle collision. If the ball has reached the paddle's height (`y + 10 >= paddleY`) AND `x` is between the left edge of the paddle (`paddleX`) and the right edge (`paddleX + paddleW`), reverse `speedY` and add 1 to `score`.

18. Level up. If `score` is at least 10 AND `level` is equal to 1, change `level` to 2 and add 2 to `speedX`. Otherwise, if `score` is at least 20 AND `level` is equal to 2, change `level` to 3 and add 2 to `speedX`.

19. Game screens. If `gameState` is `"start"`, show the text "Click to Play" at (200, 200). Otherwise, if `gameState` is `"play"`, call `moveBall()`. Otherwise, show the text "Game Over" at (200, 200).

20. Hit the target. Use `dist(x, y, targetX, targetY)` to find how far the ball is from a target. If that distance is less than 25, add 3 to `score` and move the target to a random spot: `targetX = random(width)` and `targetY = random(200)`.

21. Restart the game. Inside `function mousePressed()`, if `gameState` is NOT `"play"`, reset `score` to 0, `lives` to 3, `level` to 1, and set `gameState` to `"play"`.